"Peripheral / computer connections & Interrupt"

* I/O device & computer এর interaction
* bootstrapping ⟶ RAM এ operating system load when computer is turned on. (ROM এ program টি লেখা থাকে)

* ADD AX, BX

↓

opcode & operand in binary form

⟶ up & memory is connected to understand what does it mean by the opcode.

⟶ data transfer speed is important here.

⟶ memory ⟶ which instruction to perform

instruct ALU ⟵

CPU — memory

↕ ↕

peripheral adapter / I/O device

↕

peripherals

Data Highway

→ data bus ⟶ more data lines - more fast transfer

→ Address bus ⟶ কোন device এর সাথে connect.

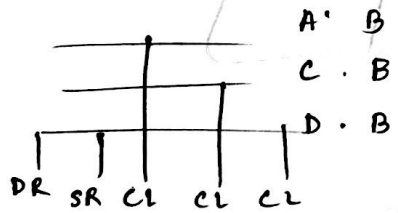
If up wants to connect with keyboard, then the address of keyboard is passed.

16 bit address ⟶ 2^{16} addresses.

(logical addresses).

→ control bus ⟶ status, timing signals.

Peripheral Adapter, reg.



All reg in ⟶ peripheral module ⟶

address comparator ⟶ up থেকে আসা address & নিজের address compare.

8255 ⟶ 60 - 63 H

match ✓ ⟶ module active via chip select

control logic ⟶ device active হবে নাকি না,

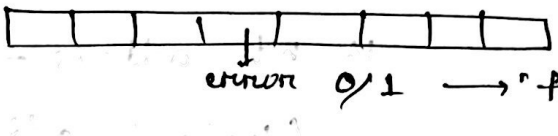
* control reg ⟶ interrupt / receive / transmit.

status " ⟶ I/O module এর status.

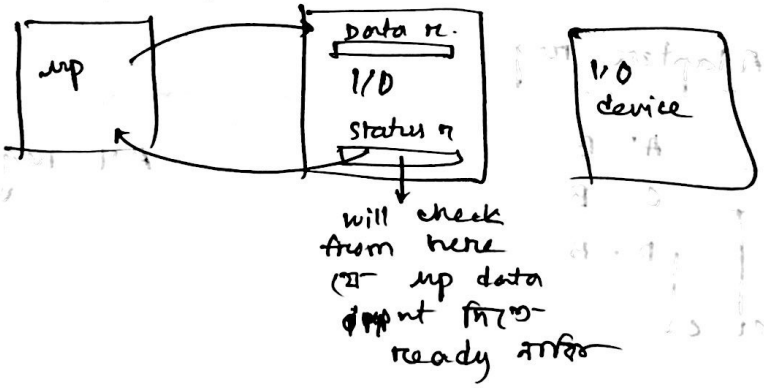
0 ⟶ data receive করতে ready কিনা

1 ⟶ " আছে নাকি

error / transmission complete / buffer overflow নাকি.



Data reg. ⟶ holds a value temporarily.



output দেয়ার ক্ষেত্রে ⟶ data reg. এ value available নাকি

avail ✓ ⟶ o/p

computer I/O operation :

* programmed I/O
* Interrupt.

Programmed I/O :

up ⟷ external device

↓

through I/O module / peripheral adapter.

output ⟶ status reg check

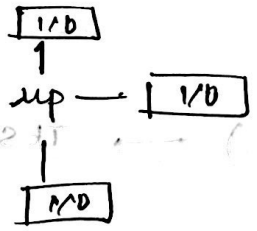
data reg available কিনা

up ⟶ gives o/p to data reg.

avail / প্রস্তুত না থাকলে wait.

up input নিতে চাইলে

Input ⟶ I/O module এ data available আছে নাকি.



⟶ poll করে buffer এ value আছে নাকি check করে sets priority to take i/p.

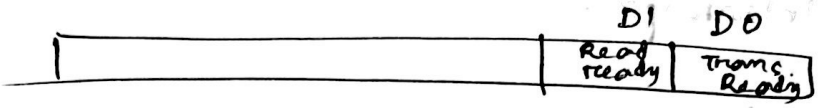
↳ sw polling.

Ex : OUT data AL

IN AL, status.

Transfer → output করার জন্য (CPU → I/O)

Receive → I/P গ্রহণ করার জন্য (I/O → CPU).




```
status  equ  0FF H
data    equ  0FE H
check : IN   AL, status.
        TEST AL, 1 H
        JZ   check.
        MOV  AL, char.
        OUT  data, AL.
```

→ status reg check

→ ~~same~~ Test Trans ready bit

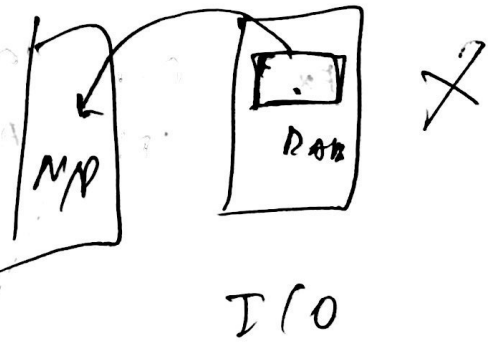
ZF → 0/1

Trans Ready.

bitwise AND → zero flag নিয়ে কাজ only (AL update করে না)

* For receive (Read ready) → TEST AL, 02 H.

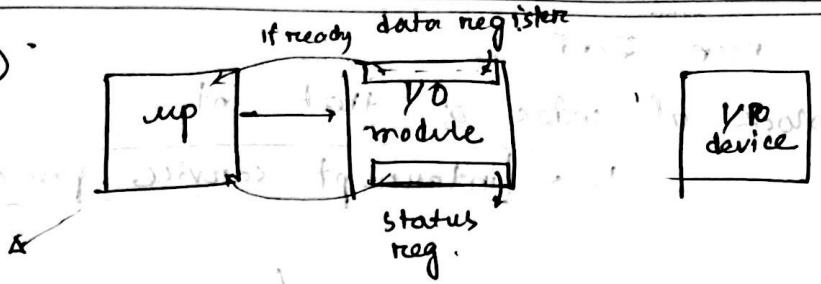
```
TEST AL, 02 H
JZ   check
~~MOV AL, char~~
IN   AL, data.
```



I/O

3.7.2025

Programmed I/O.

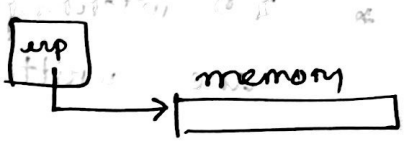


status reg. read → I/O mod → CPU → up has to wait at this time while checking that if data reg. has value.

ready or not.

ready ✓ → data reg (I/O mod) → CPU

Then, CPU → memory

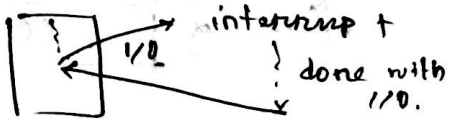


disadv : as সব step এ CPU involved, it's time consuming

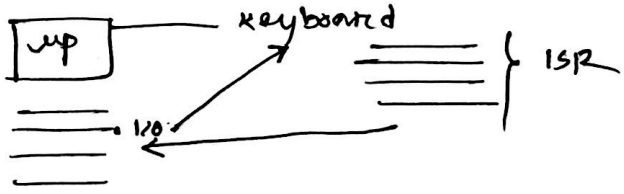
↳ soln : Interrupt.

Interrupt :

I/O detect হলো → up এর normal inst execution stop করতে বলে.



up speed ↑
I/O device speed ↓



* printer এর ক্ষেত্রে data next কখন pass করতে হবে → সে interrupt যে send করে

* Interrupt run होने
  ↳ block of codes of that int.
    ↳ **interrupt service procedure**

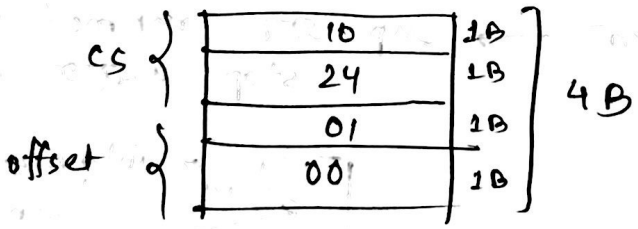
keyboard → col sel
row sel
key detect
decode key
print / mem को send

} interrupt service proc. of keyboard.

* जो interrupt आएगा, उसका ~~starting address~~ block of codes को are written in **interrupt vector table**.

* CS → 16 bits
  offset → 16 bits
  > address = (CS × 10) + offset

  If CS = 1024 offset 0100



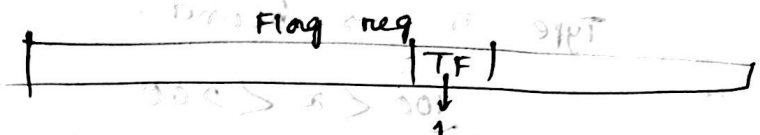
* 256 different 4B - interrupt vectors.

* 1st 32 interrupt vectors ⟶ Intel reserves. (0 - 31)

* User ⟶ 224 (32 - 255)

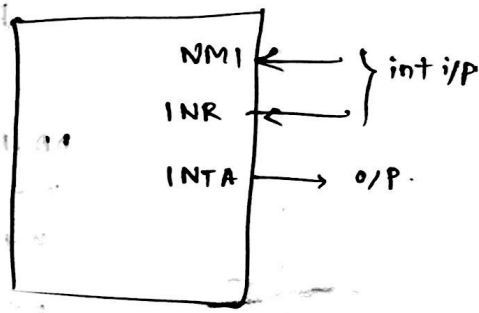
* Type 0 ⟶ divide error

- Type 1 ⟶ single - step or trap :
  (Trap bit → set হলে)
  (when we want to debug each line of code individually)



(line by line → codes will be checked).

- Type 2 ⟶ highest priority int
  Non - maskable int.
  ↓
  সব কাজ বাদ দিয়ে এই interrupt এর কাজ করবে।



INTR → normal int
INTA → int ack.

Type 3 ⟶ breakpoint of program
(1B inst) (এই পর্যন্ত run করে debugger বন্ধ হবে, then debugger যদি বলে next line execute করো, then করবে).

Type 4 ⟶ overflow

```
MOV AL, 255
ADD AL, 1.
```

↓

Flag reg ⟶ OF = 1 ⟶ overflow int will be called.

OF & TF will be used.

Type 5 ⟶ bound.

100 < x < 200

16 bit

x = 99 ⟶ int } as boundary বাইরে
x = 20 ⟶ int }

BOUND AX, DATA — memory addr. (8 bit).

DATA	01	higher limit (16 bit)
n+1	12	
n+2	23	lower n (16 bit)
n+3	44	

↓ 0112 < AX ⟶ if not then Type 5 int call.

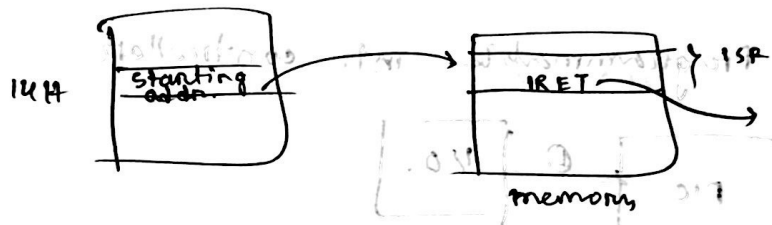
0112 < AX < 2344 ⟶ if AX boundary মধ্যে না থাকে, then Type 5 int.

Interrupt inst :

- S/W generated.
- Bound, INTO, INT n, INT 3, IRET.
- INT n → Type কোনটা

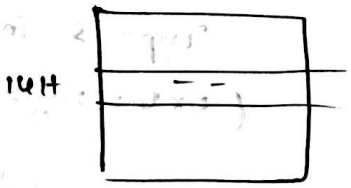
Ex : INT 5 ⟶ 5 no. INT (Type 5).

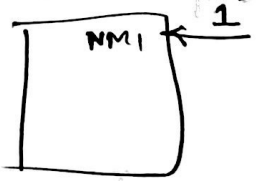
5 × 4 = 20 ⟶ 14H ← starting address of type 5 int's vector table



বাইরের থেকে int ⟶ INTR (active high)
NMI (" ")
INTA ⟶ int ack. (active low).

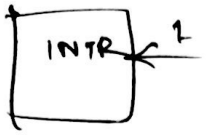
* NMI :





- up কুরাতে int আসলে
- ততক্ষণ 1 যতক্ষণ up int এর কাজ (কাজ না করে)
- IRET দেলে 0 in NMI
- NMI ⟶ parity error, power failure.

* INTR : keyboard, scanner multiple int আসতে পারে।

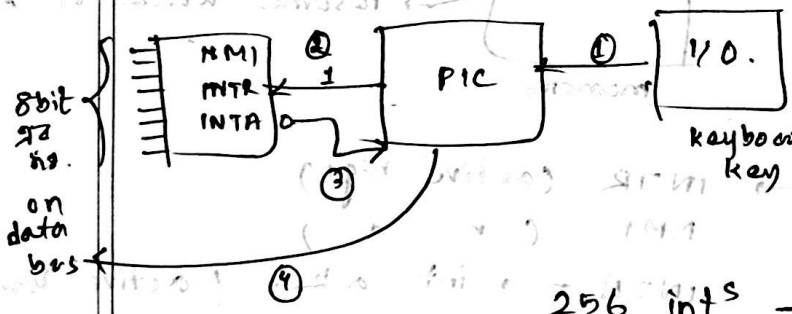


INTR → 1 হলে এই pin disable. নতুন কল আসতে পারবে না।

up will work with the int until it gets IRET; then INTR will be enabled.

how to disable / enable : TF & IF.
(trap) (interrupt)

PIC → Programmable int. controller.



256 ints → 2^8 → 8 bit দিয়ে int দিলাম।

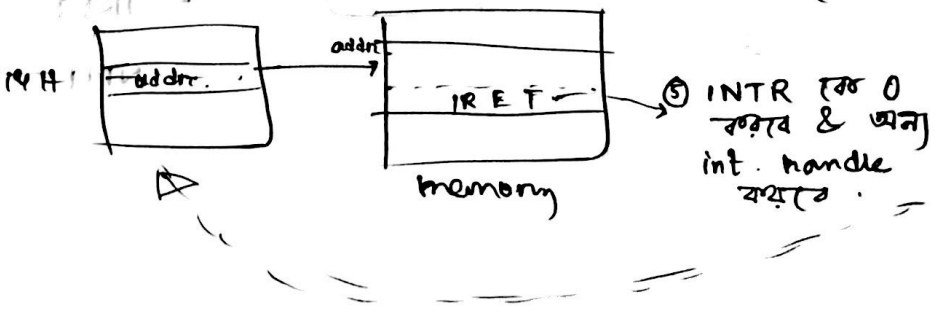
(3) ack পাওয়ার পরে up free to handle int.

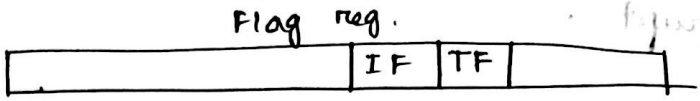
If step 4 এ, data = 5. → 00000101

Type 5 int.

($5 \times 4 = 20_{10} = 14H$)

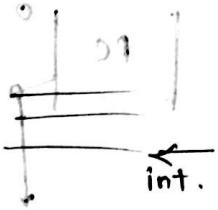
int vector table এ address





1) up executing inst^s

2) INT আসলো

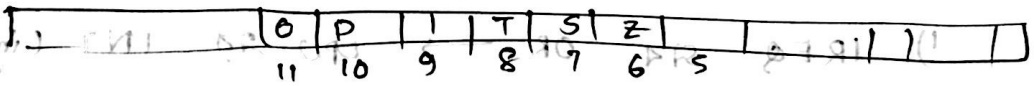


3) CS, offset value, Flag reg. ই value stack → store করতে

IF = 0
TF = 0
> then অন্য কোনো int নিত না।

after executing int (IRET আসলে), IF = 1
TF = 1

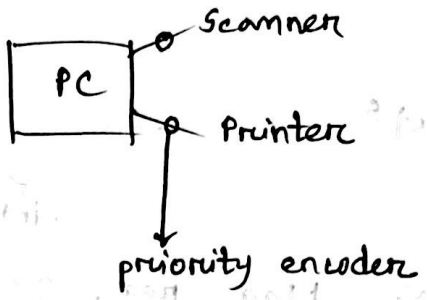
STI → set int
CLI → clear int.



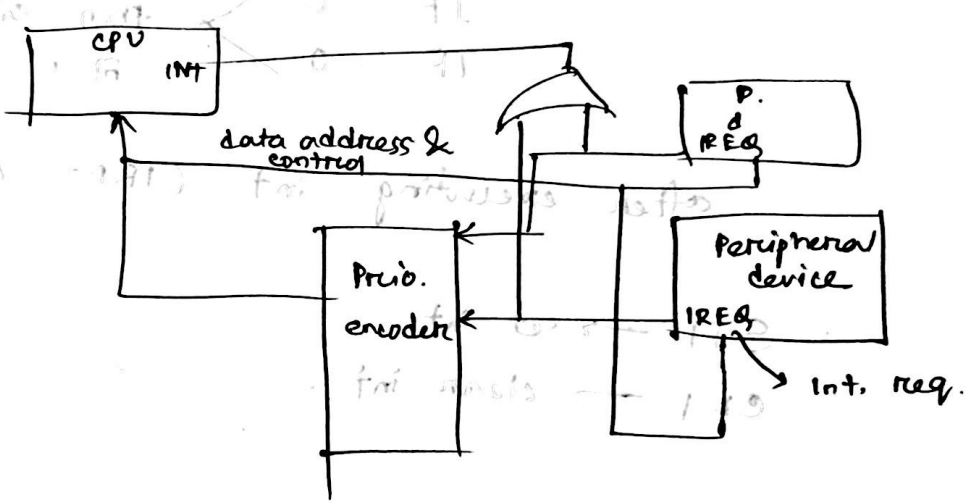
8.7.2025

Priority Interrupt :

Using P. encoder

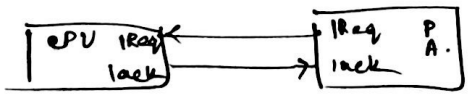


high data transfer ⟶ more priority

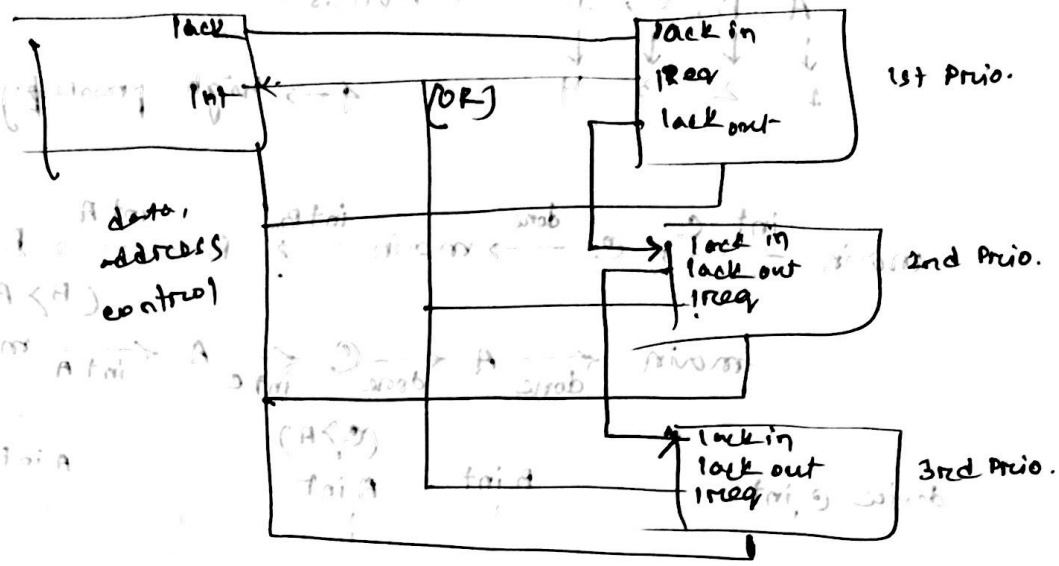


1) IREQ সব OR ⟶ CPU এর INT (interrupt আসলে বুঝায়)

2) IREQ all ⟶ Priority encoder.



Using Daisy chain



1) কার priority ↑ → Iack তার উপর dependent.
highest priority যার → তাকেই শুধু ack পাঠাবে.

2) কাজ শেষ / কাজ না থাকলে

Nested

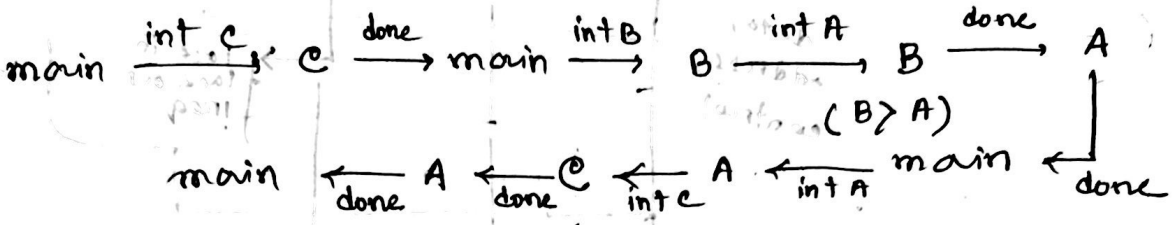
→ একটা interrupt → is interrupted by a higher priority interrupt.

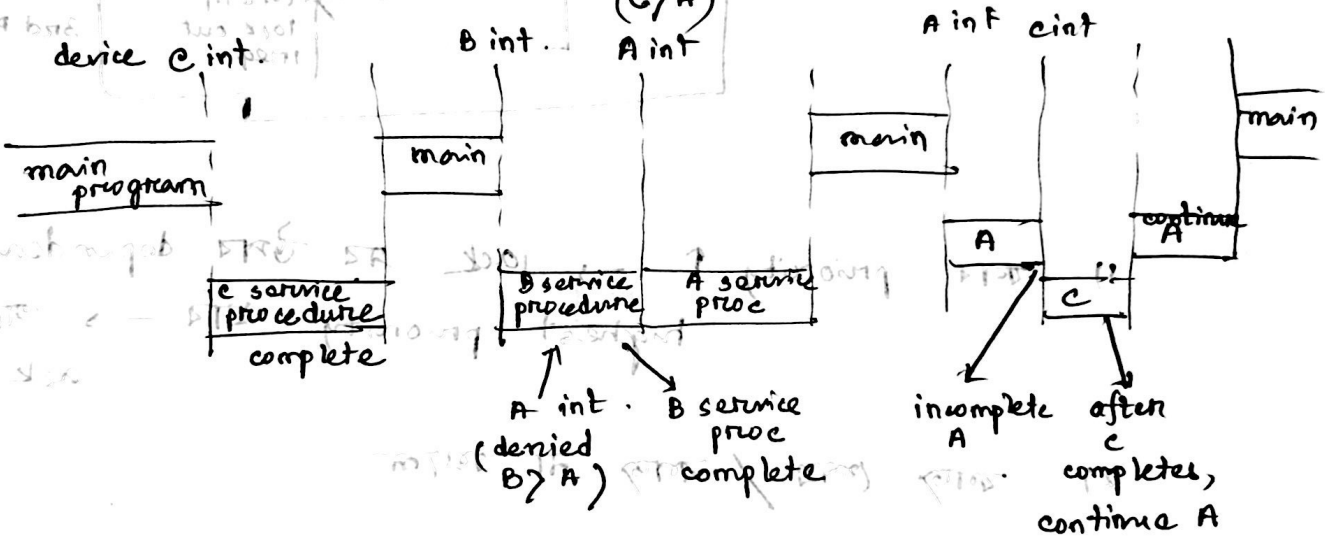
Prio → A > B > C → main

main $\xrightarrow{\text{int C}}$ C $\xrightarrow{\text{done}}$ main $\xrightarrow{\text{int B}}$ B $\xrightarrow{\text{int A}}$ A $\xrightarrow{\text{done}}$ B $\xrightarrow{\text{done}}$ main

Example :

A, B, C, D ⟶ devices.
↓ ↓ ↓ ↓
1 2 3 4 ⟶ high priority.





Part 2 → Cook & White (Book)

Block Data Transfer :

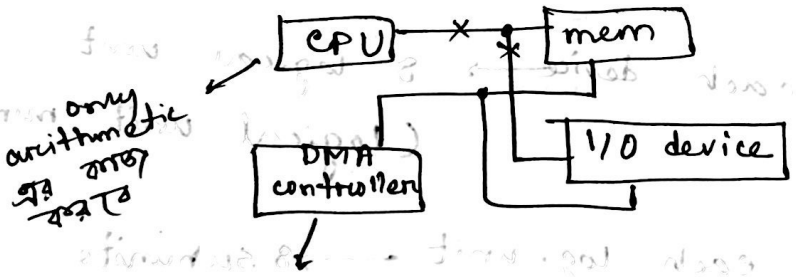
- high speed data transfer এর প্রয়োজন
- peripherals গুলোর speed, প্রায় CPU এর (high) speed এর সমান
- memory ↔ I/O device → As per step এ CPU involved that's a problem

↓ soln

DMA where CPU isn't involved.

these two are directly connected.

- DMA controller :



bus এর control থাকবে তখন DMA cont. এর কাছে।

concept :

- I/O device mem থেকে data show করতে চাচ্ছে।
- bus এ control DMA unit gain করে

This action of taking over the bus is called

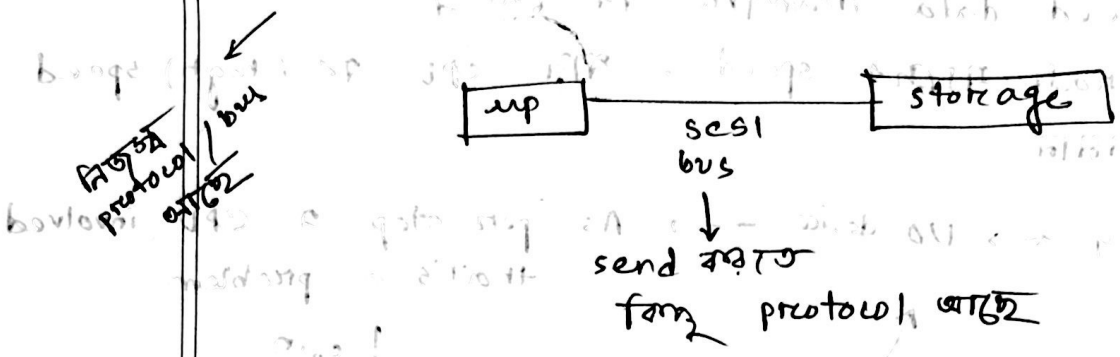
cycle stealing

- release the bus after I/O does the task

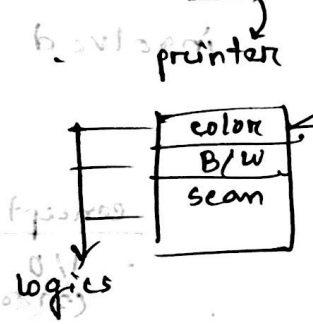
hm 5p e bvvi

cpu – storage → transfer

- SCSI (small comp. system interface)



– 8 devices can be supported by scsi bus.

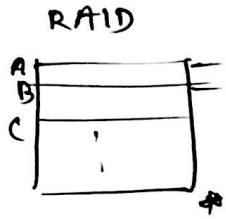


- each device → 8 logical unit.
(logical unit number)

- each log. unit → 8 subunits.

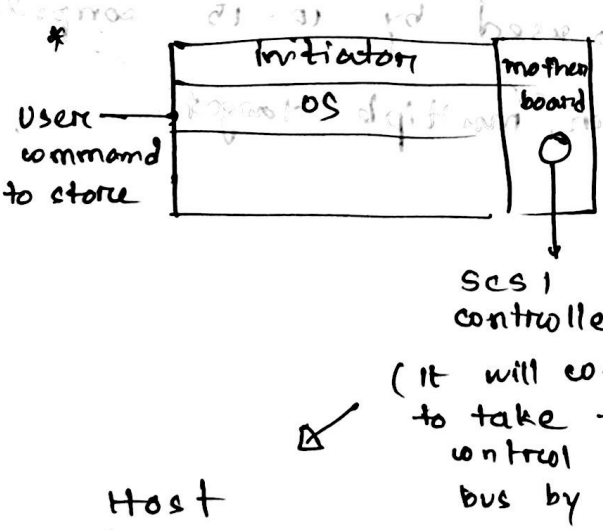
10.7.25

SCSI :



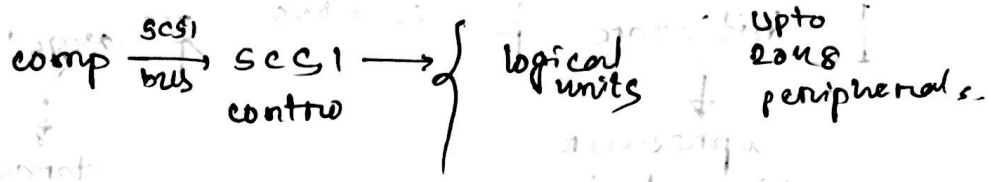
* logical unit no. → Target provide করে
↓
initiator এর command wise কাজ করে

* multiple init & multiple Target থাকতে পারে
* 8 bit bidirectional bus.
  - optional parity (error check).

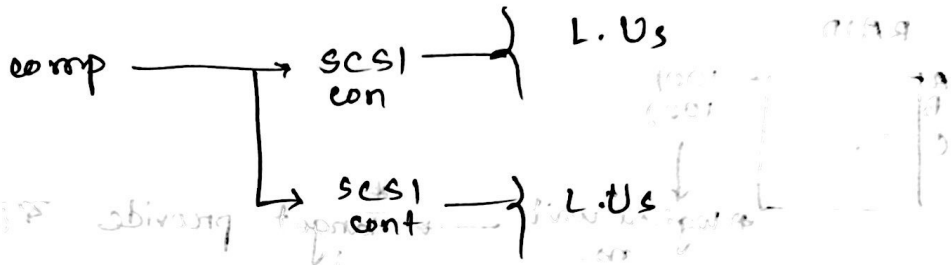


(bus এর control যে কাছে → অন্যদের জন্য disable)

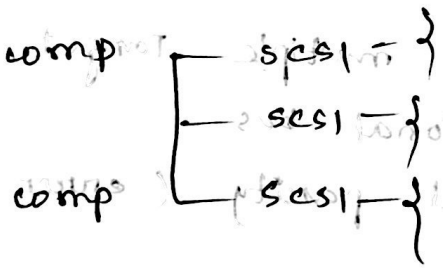
One init
One target



One init
multiple target



mul. init
target



2 or 3 storage system ⟶ used by 10 – 15 computers.

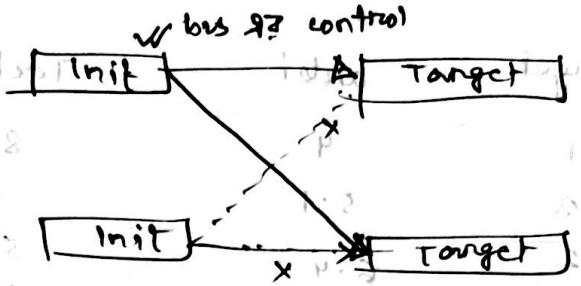
⟶ multiple initiator, multiple target.

* byte wise data আয় → SCSI protocol wise.

Protocol for SCSI bus :

Phase 1 : Arbitration

→ initiator যুক্তরাজ্য (যে বাস) করবে।



phase 2 : select target

" 3 : Info. transfer.

" 4 : Bus free. (target release busy signal).

" 5 : Reselection.

→ Target reselects the initiator as 9 B one still remaining to send.

Bus এর control still initiator এর কাছে থাকবে।

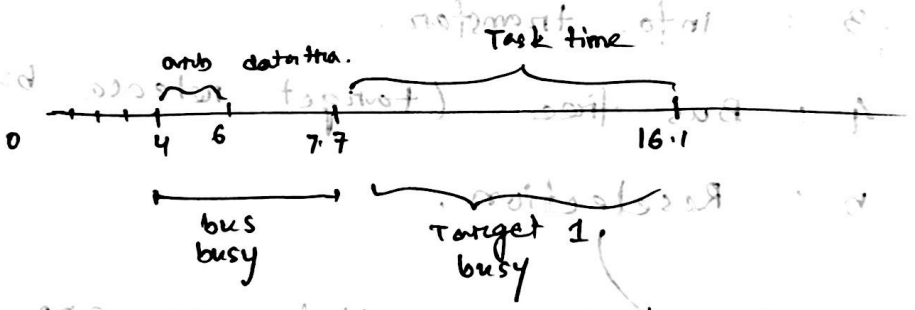
10 B data transfer করবে। (1 line এ → 1 B data transfer)

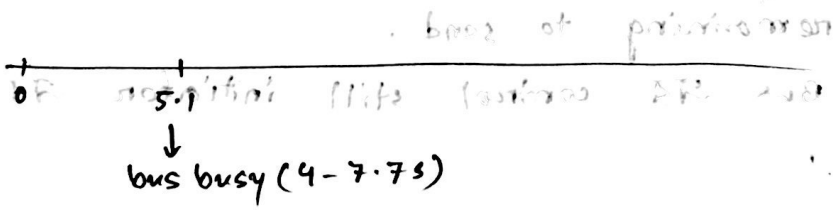
* data transfer এর পর proper জায়গায় store (Task Time).
* arbitration time at n sec. (bus এর control নেয়া)

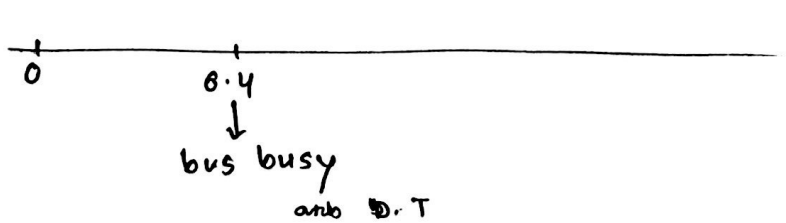
SCSI : Mathematical Problem : 1

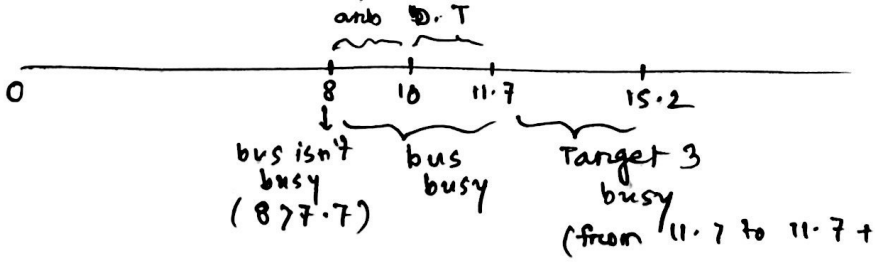
data transfer time = 1.7 sec.
arbi + sel / resel time = 2 sec.

	Init.	Target	arbi	Task Time
①	2	1	4	8.4
②	1	3	5.1	5
③	1	2	6.4	5.8
④	1	3	8	3.5
	2	2	8.2	4.9

① 

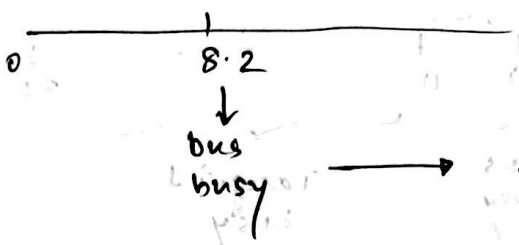
② 

③ 

④ 

(from 11.7 to 11.7 + 3.5 = 15.2s)

(5)



⟶ operation perform হবে না।

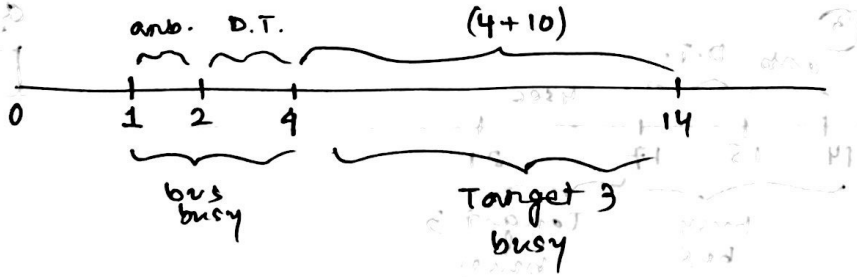
but if Target is busy. then সে initiator কে queue তে রাখবে।

Problem 2

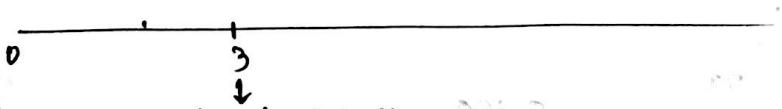
D. T. ম = 2 sec.
arbit + set/reset t= 1 s.

Init.	Tar	arbi	Task
2	3	1	10
1	2	3	8
1	3	6	4
3	1	8	20
3	3	12	9

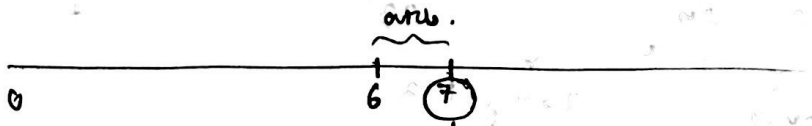
2 → 3 (1)



1 → 2 (2)



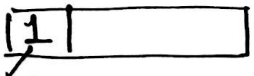
1 → 3 (3)



↳ target 3 busy from 4 to 14 sec.

bus free

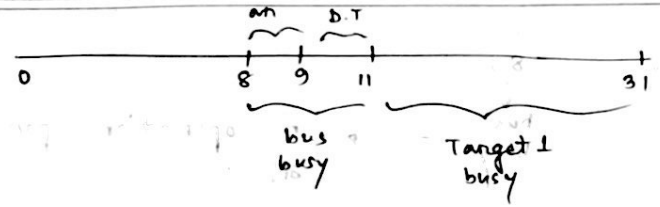
Target 3's queue :



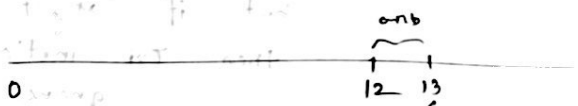
Init in queue.

If ⑤ 3 - 3 — 15 → queue theke value neya kaj ③, 1 → 3 er kaj age hobe.
then, 3 → 3 er kaj hobe.

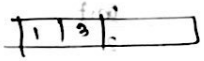
3 → 1 ④



3 → 3 ⑤



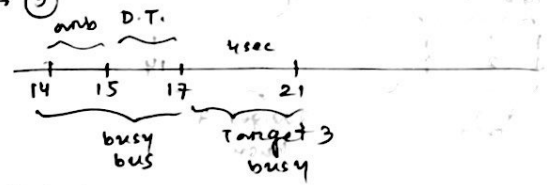
Target 3 busy till 14
So, in queue



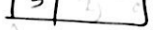
initiators ① & ③ in queue

(after) 14th sec :

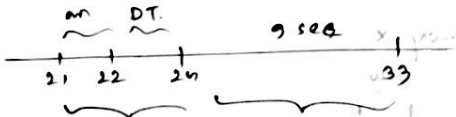
① → ③



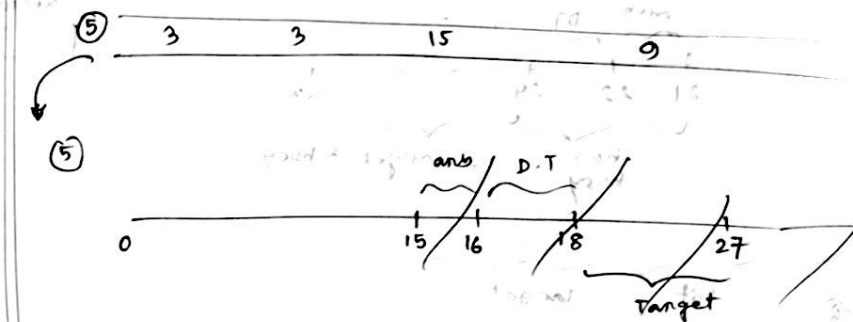
Q :



(after) 21st sec :

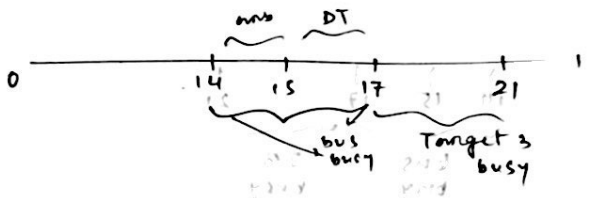


Q :

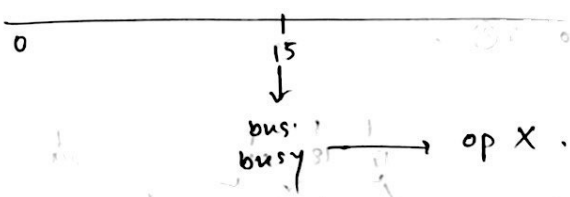


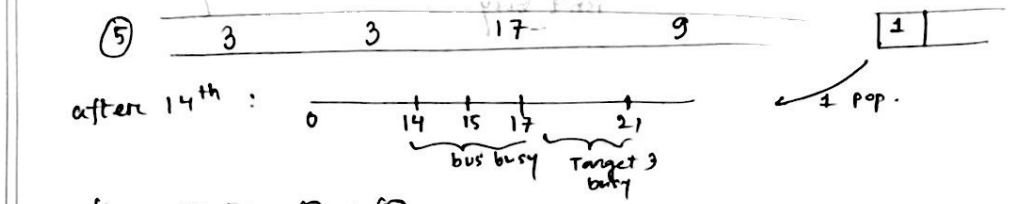
after 14 th : pop queue থেকে ①.

1 → 3

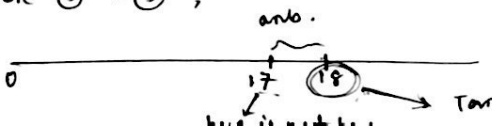


3 → 3

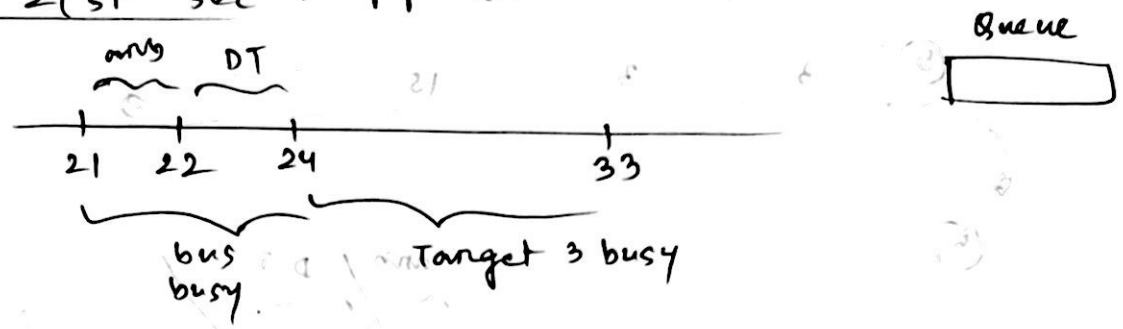




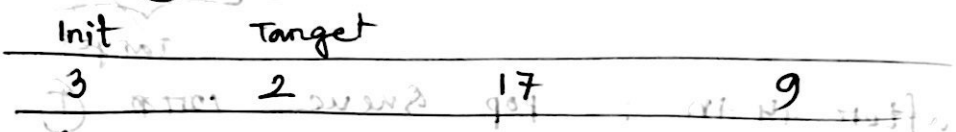
~~after 17~~ For ③ → ③ ,



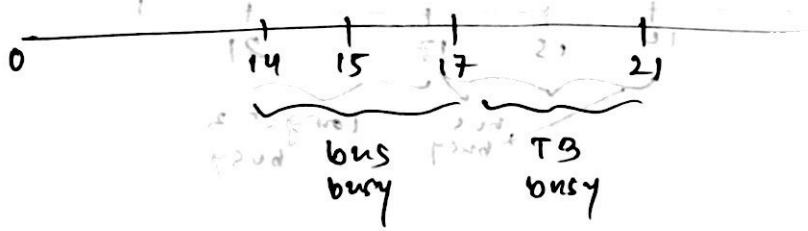
after 21st sec : pop 1(3) from Queue.



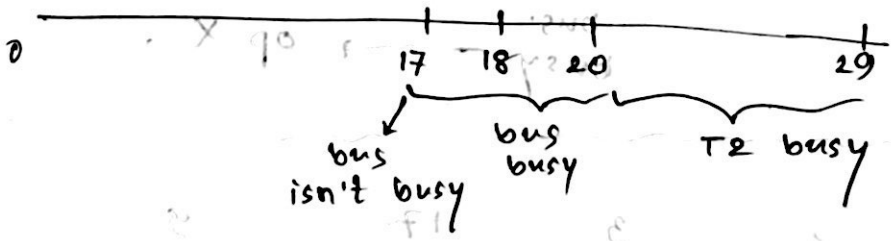
⑤



after 14th



for 1③ to T②,



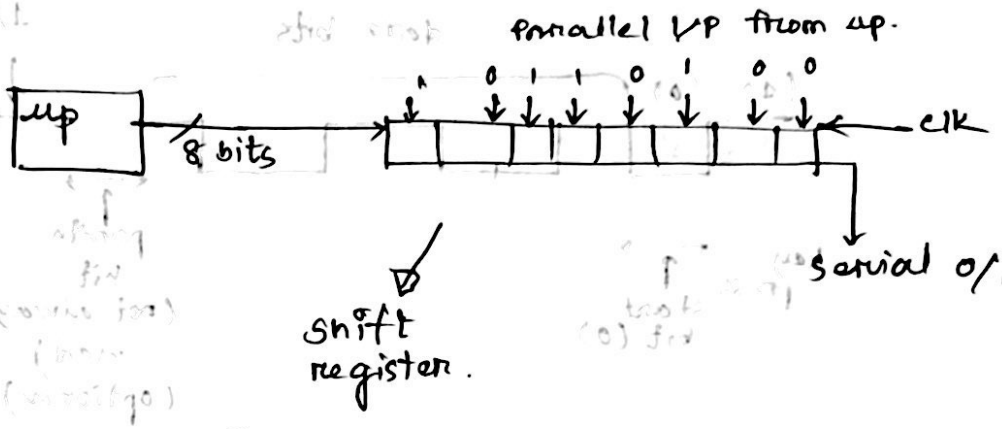
both → synchronous.
(Tx → Rx → clk same).

Parallel to Serial Interface :

Scenario : printer → 1 bit করে data নেয়

up → 8 " " " "

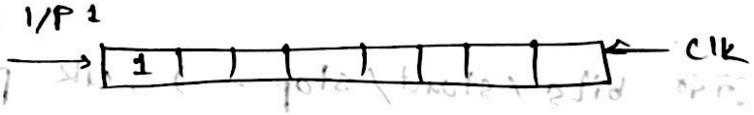
So, মাঝে interfacing needed.



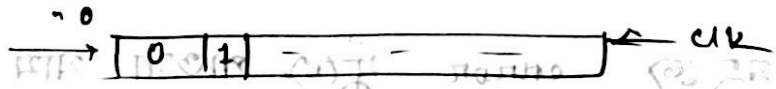
Serial to Parallel :

Scanner → 1 bit করে data দেয়।

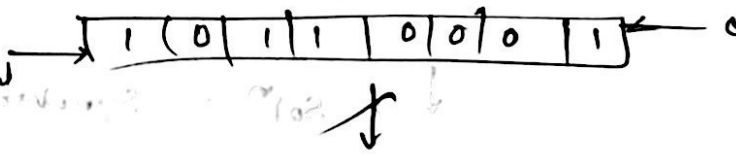
1) Serial I/P 1



2) " " 0



after 8 clk prise :



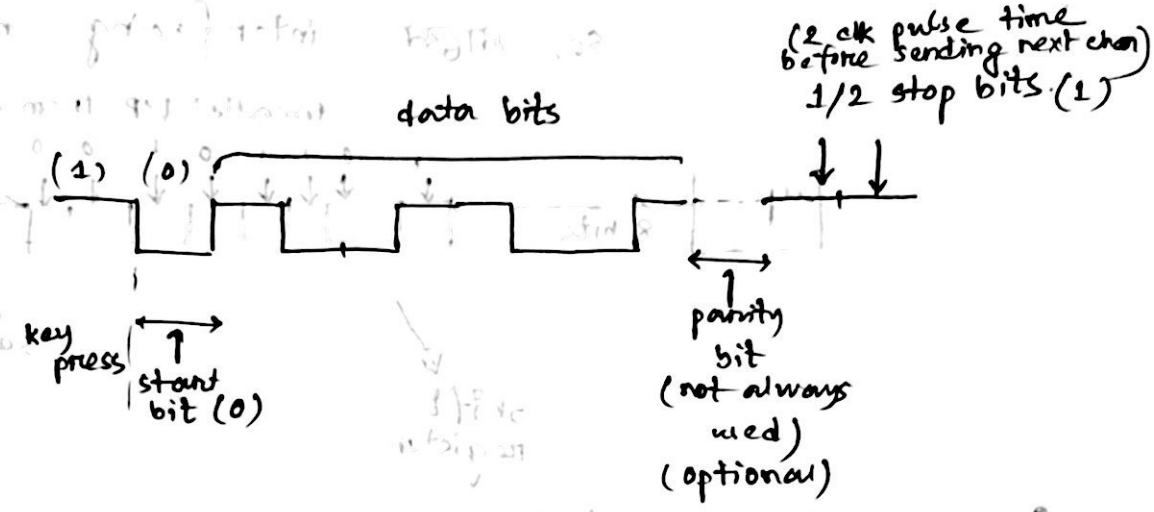
Asynchronous :

keyboard data পাঠানো বন্ধ করার পর next char যাবে → not equal

idle ⟶ mark (1) ⟶ signal line ↑ 1.

keyboard ↑ value আসলে,

start bit / space / 0 in start line.



* S & R ⟶ not synchronized at char. level.

disadv :

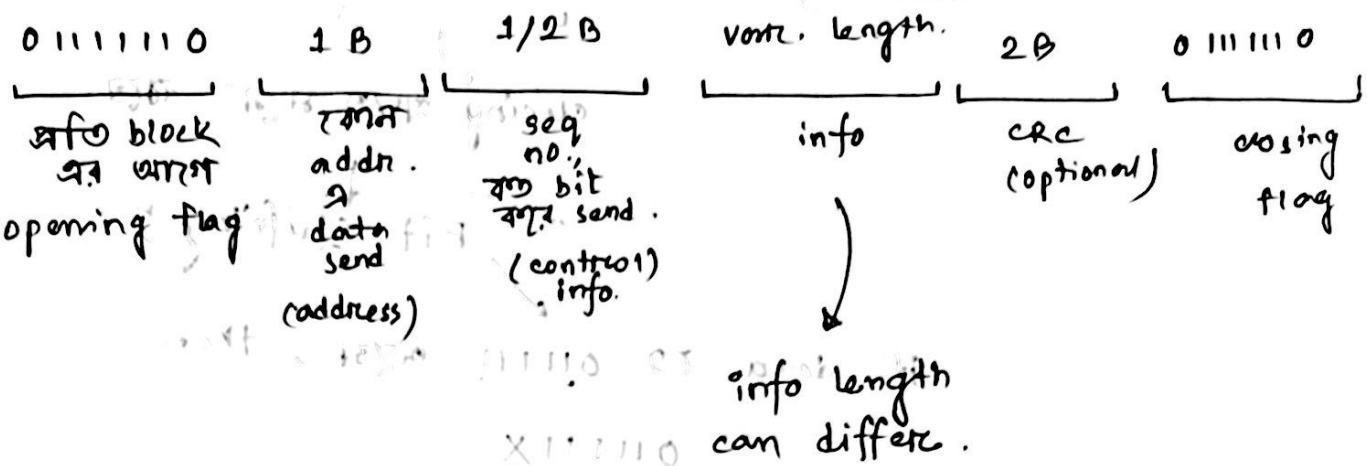
→ অনেক bits (start / stop ...) ; clk pulse ↑

→ সহজে error খুঁজে পাওয়া যায়।

↳ Sol^n : Synchronous Trans.

- clk + data signal.
- আলাদা protocols
  - one of them
    HDLC
    (high level Data Link control)

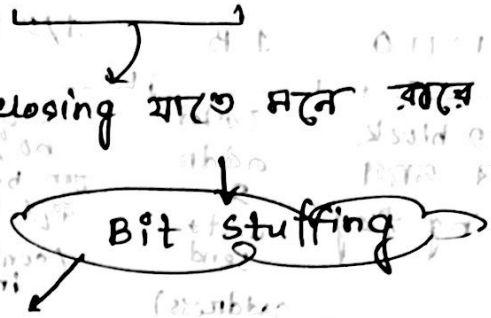
HDLC format :


01111110
1 B
1/2 B
vary. length.
2B
01111110
প্রতি block এর আগে opening flag
কোন addr. এ data send (address)
seq no.; কত bit কয়বার send. (control) info.
info
CRC (optional)
closing flag
info length can differ.

15.7.2025

HDLC ⟶ disadv :

data/info ⟶ ... 01111110



if data তে 011111 আসে, then

011111X

↓

0 দিয়ে stuff.

(2) info given
transmit কিভাবে করবে।

(3) receiver ⟶ given
original data কি ?

org ⟶ 01111110|01111110

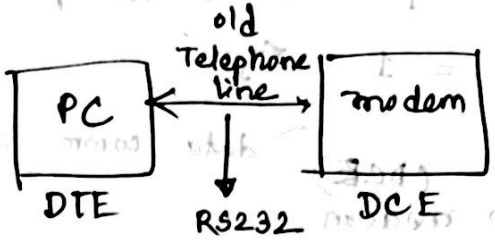
↓

01111110 101111 [0] 10

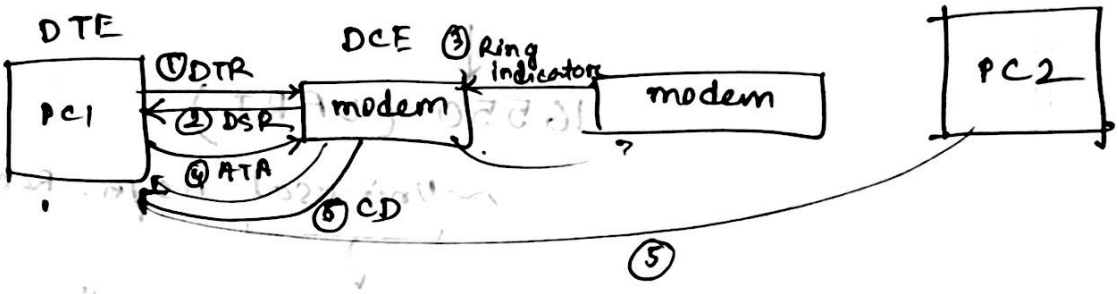
stuffed bit.

RS232 standard :

computer to computer connection, we need modem.



* all pc uses it's own modem.



DT. যার জন্য ready

1) Data Terminal Ready.
2) " Set "

3) Ring indicator → normally 1
0 → communication করার try করছে (some PC)
(PC2 থেকে request আসলো)

4) If PC1 free → ATA

5) PC1 to PC2 connect via modems.

6) CD → carrier detect.

7) active low = 0
RTS → req. to send. (PC2 data send করার)
CTS → clear " " (done